

UD3.2: Entornos virtuales, pip y .gitignore — Guía práctica

Módulo: ESP — Entornos y Sintaxis en Python (5098)

Unidad: UD3 — Entornos y Control de Versiones

Criterios: CE3_b (selección de entornos según desarrollo), CE3_d (comparar editores/herramientas)

Índice de contenidos

1. [El problema: dependencias](#)
2. [pip: el gestor de paquetes](#)
3. [Entornos virtuales con venv](#)
4. [requirements.txt](#)
5. [Conectar con Git: .gitignore](#)
6. [Tabla resumen de comandos](#)

1. El problema: dependencias

Cuando se programa en Python, es habitual necesitar **paquetes externos** (librerías que no vienen incluidas con Python). Por ejemplo, `requests` para hacer peticiones HTTP o `flask` para crear una web.

Instalar paquetes a nivel global: el problema

Si se instalan todos los paquetes directamente en el Python del sistema, pueden surgir conflictos:

Proyecto A necesita `requests 2.28`

Proyecto B necesita `requests 2.31`

- Solo puede haber UNA versión instalada a la vez
- Actualizar para B puede romper A

Situación	Consecuencia
Dos proyectos necesitan versiones distintas del mismo paquete	Solo funciona uno de los dos
Se instalan paquetes de prueba que ya no se necesitan	El Python del sistema se llena de basura
Se cambia de ordenador o se trabaja en equipo	No se sabe qué paquetes hay que instalar

La solución

Cada proyecto tiene su **propia carpeta** con su propia copia de Python y sus propios paquetes instalados. Eso es un **entorno virtual**.

2. pip: el gestor de paquetes

`pip` es la herramienta que instala, desinstala y gestiona paquetes de Python. Viene incluida con Python.

Comandos básicos

Comando	Qué hace	Ejemplo
<code>pip install paquete</code>	Instala un paquete	<code>pip install requests</code>
<code>pip install paquete==versión</code>	Instala una versión concreta	<code>pip install requests==2.28.0</code>
<code>pip uninstall paquete</code>	Desinstala un paquete	<code>pip uninstall requests</code>
<code>pip list</code>	Lista todos los paquetes instalados	<code>pip list</code>
<code>pip show paquete</code>	Muestra información de un paquete	<code>pip show requests</code>
<code>pip freeze</code>	Lista paquetes en formato para	<code>pip freeze</code>

Comando	Qué hace	Ejemplo
	requirements.txt	

Ejemplo: pip show

```
$ pip show requests
Name: requests
Version: 2.31.0
Summary: Python HTTP for Humans.
Requires: charset-normalizer, idna, urllib3, certifi
```

Nota: pip por defecto instala paquetes **a nivel global** (para todo el sistema). Por eso necesitamos entornos virtuales.

3. Entornos virtuales con venv

Qué es un entorno virtual

Un entorno virtual es una **carpeta** dentro del proyecto que contiene:

- Una copia del intérprete de Python
- Su propia carpeta de paquetes (independiente del sistema)
- Scripts para activar y desactivar el entorno

Cada proyecto puede tener su propio entorno virtual con sus propios paquetes, sin afectar a otros proyectos.

Crear un entorno virtual

Abrir una terminal en la **carpeta del proyecto** y ejecutar:

```
python -m venv .venv
```

Esto crea una carpeta `.venv` dentro del proyecto. El nombre `.venv` es una convención (el punto la oculta en Linux/Mac).

En VSCode: abrir la terminal integrada con `ctrl+`` y ejecutar el comando ahí. Así la terminal ya está en la carpeta del proyecto.

Activar el entorno virtual

Sistema	Comando
Windows (cmd)	<code>.venv\Scripts\activate</code>
Windows (PowerShell)	<code>.venv\Scripts\Activate.ps1</code>
Linux / Mac	<code>source .venv/bin/activate</code>

Al activar el entorno, el **prompt** de la terminal cambia para indicar que estamos dentro:

```
(.venv) C:\Users\alumno\mi_proyecto>
```

Importante: a partir de ahora, cualquier `pip install` instalará paquetes **solo dentro de .venv**, sin afectar al sistema.

VSCode lo detecta automáticamente

Cuando se crea un `.venv` en la carpeta del proyecto, VSCode lo detecta y pregunta si se quiere usar como intérprete. Hacer clic en **"Yes"**. A partir de ese momento:

- La terminal integrada activa el entorno automáticamente
- El intérprete de Python que usa VSCode es el del entorno virtual

Si no lo detecta, seleccionarlo manualmente: `ctrl+shift+P` → "Python: Select Interpreter" → elegir el de `.venv`.

Desactivar el entorno virtual

```
deactivate
```

El prompt vuelve a la normalidad. Los paquetes del entorno siguen instalados en `.venv`, simplemente ya no se usan.

Ejercicio: crear y usar un entorno virtual

1. Crear una carpeta de proyecto:

```
mkdir mi_proyecto
cd mi_proyecto
```

2. Crear el entorno virtual:

```
python -m venv .venv
```

3. Activar el entorno:

- Windows: `.venv\Scripts\activate`
- Linux/Mac: `source .venv/bin/activate`
- Comprobar que el prompt muestra `(.venv)`

4. Instalar un paquete:

```
pip install requests
```

5. Verificar la instalación:

```
pip list
```

Salida esperada:

Package	Version
certifi	2024.x.x
charset-normalizer	3.x.x
idna	3.x
pip	24.x
requests	2.31.0
urllib3	2.x.x

6. Probar que funciona — crear un archivo `prueba.py` :

```
import requests

respuesta = requests.get("https://httpbin.org/get")
print(f"Status: {respuesta.status_code}")
print(f"IP: {respuesta.json()['origin']}")
```

7. Ejecutar:

```
python prueba.py
```

4. requirements.txt

El problema

El entorno virtual (`.venv`) es una carpeta pesada (puede ocupar cientos de MB) y es **específica del sistema operativo**. No tiene sentido copiarla ni subirla a git.

Pero entonces, ¿cómo sabe otra persona qué paquetes necesita el proyecto?

La solución: requirements.txt

Un archivo de texto que lista todos los paquetes necesarios con sus versiones:

```
certifi==2024.8.30
charset-normalizer==3.3.2
idna==3.8
requests==2.31.0
urllib3==2.2.2
```

Generar requirements.txt

```
pip freeze > requirements.txt
```

`pip freeze` lista los paquetes instalados en formato `paquete==versión` . El `>` redirige la salida a un archivo.

Instalar desde requirements.txt

```
pip install -r requirements.txt
```

Instala **todos** los paquetes listados en el archivo, con las versiones exactas.

El flujo completo

Desarrollador A (crea el proyecto):

1. `python -m venv .venv`
2. Activa el entorno
3. `pip install requests flask ...`
4. `pip freeze > requirements.txt`
5. Sube requirements.txt a git (NO sube .venv)

Desarrollador B (clona el proyecto):

1. `git clone <url>`
 2. `python -m venv .venv`
 3. Activa el entorno
 4. `pip install -r requirements.txt`
- Tiene exactamente los mismos paquetes

Ejercicio: recrear un entorno desde requirements.txt

1. Asegurarse de que el entorno está activado y tiene paquetes:

```
pip list
```

2. Generar el archivo:

```
pip freeze > requirements.txt
```

3. Ver el contenido:

```
type requirements.txt    (Windows)
cat requirements.txt      (Linux/Mac)
```

4. Desactivar y borrar el entorno:

```
deactivate
```

- Windows: `rmdir /s /q .venv`
- Linux/Mac: `rm -rf .venv`

5. Recrear desde cero:

```
python -m venv .venv
```

Activar el entorno y verificar que está vacío:

```
pip list
```

6. Instalar desde requirements.txt:

```
pip install -r requirements.txt
```

7. Verificar que todo vuelve a estar:

```
pip list  
python prueba.py
```

5. Conectar con Git: .gitignore

El problema

Al hacer `git add .` o al ver los cambios en VSCode, la carpeta `.venv` aparece como archivos nuevos — **cientos o miles de archivos**. No queremos subirlos al repositorio porque:

Razón	Detalle
Es enorme	<code>.venv</code> puede tener miles de archivos y cientos de MB
Es específico del PC	Un <code>.venv</code> de Windows no funciona en Linux y viceversa
Se puede recrear	Con <code>requirements.txt</code> cualquiera puede recrear el entorno

Qué es .gitignore

Un archivo llamado `.gitignore` en la raíz del proyecto que indica a Git qué archivos y carpetas **debe ignorar**. Cada línea es un patrón:


```
# Entorno virtual
.venv/

# Archivos compilados de Python
__pycache__/
*.pyc

# Variables de entorno (contraseñas, claves API)
.env

# Configuración del IDE (opcional)
.vscode/
```

Patrón	Qué ignora
.venv/	La carpeta .venv y todo su contenido
__pycache__/	Carpetas de caché de Python
*.pyc	Archivos compilados de Python
.env	Archivo de variables de entorno
# texto	Comentario (Git lo ignora)

Cómo funciona

1. Git lee .gitignore antes de decidir qué archivos "ve"
2. Los archivos que coincidan con un patrón son **invisibles** para Git
3. No aparecen en `git status`, no se añaden con `git add`.
4. El propio .gitignore **sí se sube** a git (es parte del proyecto)

Verificar en VSCode

Después de crear .gitignore :

- La carpeta .venv deja de aparecer en el panel de **Source Control** (icono de rama)
- Los archivos ignorados aparecen en gris en el explorador de archivos (si están visibles)

El flujo completo de un proyecto Python con Git

- | | |
|--|---|
| 1. <code>git clone <url></code> | ← Descargar el proyecto |
| 2. <code>python -m venv .venv</code> | ← Crear entorno virtual |
| 3. Activar el entorno | ← <code>.venv\Scripts\activate</code> (Windows) |
| 4. <code>pip install -r requirements.txt</code> | ← Instalar dependencias |
| 5. Trabajar en el proyecto... | |
| 6. <code>pip freeze > requirements.txt</code> | ← Si se añaden paquetes nuevos |
| 7. <code>git add requirements.txt</code> | ← Subir el archivo de requisitos |
| 8. <code>git commit -m "Añadir nueva dependencia"</code> | |

Lo que **SÍ** se sube a git:

- Código fuente (`.py`)
- `requirements.txt`
- `.gitignore`

Lo que **NO** se sube a git:

- `.venv/`
- `__pycache__/`
- `.env`

Ejercicio: crear `.gitignore` y verificar

1. Inicializar un repositorio (si no existe):

```
git init
```

2. Comprobar el estado **antes** de crear `.gitignore` :

```
git status
```

→ Se ven miles de archivos de `.venv/`

3. Crear el archivo `.gitignore` con el siguiente contenido:

```
# Entorno virtual
.venv/

# Archivos compilados de Python
__pycache__/*
*.pyc

# Variables de entorno
.env
```

4. Comprobar el estado **después**:

```
git status
```

→ .venv/ ya no aparece

5. Verificar en VSCode: abrir el panel Source Control y confirmar que solo aparecen los archivos del proyecto.

6. Hacer el primer commit:

```
git add .
git commit -m "Primer commit con .gitignore y requirements.txt"
```

6. Tabla resumen de comandos

pip

Comando	Qué hace
pip install paquete	Instala un paquete
pip install paquete==1.0	Instala una versión concreta
pip uninstall paquete	Desinstala un paquete
pip list	Lista los paquetes instalados
pip show paquete	Información detallada de un paquete
pip freeze	Lista paquetes en formato requirements

Comando	Qué hace
<code>pip freeze > requirements.txt</code>	Genera el archivo de requisitos
<code>pip install -r requirements.txt</code>	Instala desde el archivo de requisitos

Entorno virtual (venv)

Comando	Qué hace
<code>python -m venv .venv</code>	Crea un entorno virtual en la carpeta <code>.venv</code>
<code>.venv\Scripts\activate</code>	Activa el entorno (Windows cmd)
<code>.venv\Scripts\Activate.ps1</code>	Activa el entorno (Windows PowerShell)
<code>source .venv/bin/activate</code>	Activa el entorno (Linux/Mac)
<code>deactivate</code>	Desactiva el entorno

Git (.gitignore)

Acción	Cómo
Crear <code>.gitignore</code>	Crear archivo en la raíz del proyecto
Ignorar carpeta	Añadir <code>carpeta/</code> al archivo
Ignorar archivos por extensión	Añadir <code>*.extensión</code> al archivo
Comentario	Línea que empieza por <code>#</code>
Ver qué ignora git	<code>git status --ignored</code>

Flujo completo de un proyecto nuevo

Paso	Comando	Solo una vez / Siempre
1	<code>git init</code>	Una vez
2	Crear <code>.gitignore</code>	Una vez
3	<code>python -m venv .venv</code>	Una vez

Paso	Comando	Solo una vez / Siempre
4	Activar el entorno	Cada vez que se abre el proyecto
5	<code>pip install paquete</code>	Cuando se necesite un paquete nuevo
6	<code>pip freeze > requirements.txt</code>	Después de instalar paquetes
7	<code>git add .</code> y <code>git commit</code>	Cuando se quiera guardar cambios

Flujo para clonar un proyecto existente

Paso	Comando
1	<code>git clone <url></code>
2	<code>cd nombre_proyecto</code>
3	<code>python -m venv .venv</code>
4	Activar el entorno
5	<code>pip install -r requirements.txt</code>

Recursos adicionales

- Documentación oficial de venv: docs.python.org/3/library/venv.html
- pip User Guide: pip.pypa.io/en/stable/user_guide
- Patrones de .gitignore para Python: github.com/github/gitignore/blob/main/Python.gitignore